Projet Génie Logiciel

Projet LambdaSys

Année 2025-2026

EL MAKHLOUFI Tarik
BOURGER Léo
JAZIRI Jihène

Encadré par : **FRANCISCI Dominique**

UFR Sciences & Techniques
Université de Toulon

Table des matières

1 Introduction **3**
1.1 Présentation du projet 3
1.2 Approche du sujet 3

2 Présentation de l'équipe **3**
2.1 Planning 4

3 Spécifications Logicielles **5**
3.1 Exigences Fonctionnelles (Métier) 5
3.2 Exigences d'Interface et Non-Fonctionnelles 5

4 Conception et Maquettage **5**
4.1 Maquettage de l'IHM 5
4.2 Modélisation UML 5
4.2.1 Axe Fonctionnel (Cas d'utilisation) 5
4.2.2 Axe Statique (Diagramme de Classes) 9
4.2.3 Axe Dynamique (Diagramme de Séquence) 10

5 Implémentation et Bilan Technique **11**
5.1 Choix techniques : Introspection et Généricité 11
5.2 Problèmes rencontrés et solutions 12

6 Difficultées rencontrées **12**

7 Conclusion **13**

8 Annexe **14**

1. Introduction

1.1 Présentation du projet

L'objectif du projet LambdaSys et de créer un logiciel générique permettant de modéliser des systèmes techniques.

Le logiciel doit être capable de pouvoir générer des jeux de données pour ces systèmes et doit pouvoir également gérer les anomalies dans le but de pouvoir entrainer des modèles d'IA discriminantes ou génératives.

1.2 Approche du sujet

Pour répondre aux exigences du sujet, il est nécéssaire de structurer le projet dès le départ afin de construire de manière optimale le logiciel.

Il est absolument nécéssaire de conserver l'idée générative du logiciel donc dans un premier temps il faut se concentrer sur les exigences.

Tout d'abord il faut concevoir une petite maquette pour avoir une première approche du logiciel afin de pouvoir établir les différentes exigences logicielles. Après avoir clairement définit ces exigences il faut concevoir l'axe fonctionnel en commençant par le diagramme de cas d'utilisation.

Une fois le diagramme conçu, il faut faire un diagramme de séquence qui répond aux exigences du logiciel.

Ensuite vient l'axe statique avec le diagramme de classe qui est l'étape cruciale qui nous servira de base pour la partie codage du logiciel.

Une fois que tout l'UML est finalisé, on pourra commencer la construction du logiciel qui sera réalisé en Python Orienté Objet.

Enfin pour avoir un suivi complet du projet nous utiliserons Github.

2. Présentation de l'équipe

Ce projet sera réalisé par 3 collaborateurs :

- Bourger Léo

- El makhloufi Tarik

- Jaziri Jihène

Il est important de répartir de façon équitable les différentes tâche du projet. Donc il n'y a pas de rôle défini pour chaque collaborateur.

Pour chaque séance de projet on définit clairement les tâches du jour et on les répartit entre chaque personne afin d'avoir un suivi parallèle et au fur et à mesure de la séance, on réalise une mise en commun pour un suivi plus fluide .

2.1 Planning

Voici le planning complet du projet représentant les tâches de chaque personne durant les séances de TP :

Date	**Membre(s)**	**Tâche réalisée**
02/02/2026	Toute l'équipe	Construction de l'équipe
03/02/2026	Toute l'équipe	Analyse du cahier des charges et Conception des Exigences
10/02/2026	Léo	Finalisation des exigences et début de l'Axe Fonctionnel
10/02/2026	Tarik	Création du dépot Git et début de l'Axe Fonctionnel
10/02/2026	Jihène	Réalisation de la maquette du logiciel et début de l'Axe Fonctionnel
17/02/2026	Toute l'équipe	Modélisation UML et Avancement de l'axe Fonctionnel
03/03/2026	Tarik	Mise en commmun de l'Axe Fonctionnel et début de l'Axe Statique
03/03/2026	Léo	début de l'Axe Statique
03/03/2026	Jihène	Création de la première fenêtre Python
10/03/2026	Toute l'équipe	Avancement de l'Axe Statique et début de l'Axe Dynamique
17/03/2026	Jihène	Finalisation de l'axe Statique et Dynamique
17/03/2026	Léo	Finalisation de l'axe Statique et Dynamique
17/03/2026	Tarik	Finalisation de l'axe Statique et Début de l'implémentation en Python
24/03/2026	Toute l'équipe	Avancement du logiciel en python
31/03/2026	Toute l'équipe	Avancement du logiciel en python
07/04/2026	Tarik	Avancement du Compte rendu
07/04/2026	Jihène	Création du PowerPoint
07/04/2026	Léo	Mise à jour de l'UML en fonction de l'implementation en Python (Diagramme de classe)
14/04/2026	Toute l'équipe	Finalisation de l'implémentation Python

Table 1 – Extrait du carnet de bord du projet LambdaSys

3. Spécifications Logicielles

Afin de garantir la généricité et la robustesse du logiciel LambdaSys, nous avons établi un cahier des charges précis, divisé en trois catégories d'exigences.

3.1 Exigences Fonctionnelles (Métier)

- **MODELISATION :** Le logiciel doit permettre d'importer des modèles existants via des fichiers externes contenant les lois physiques du système. L'utilisateur doit pouvoir visualiser l'arborescence des composants de ce système.
- **CAPTEUR :** Chaque composant peut disposer de capteurs virtuels configurables (Type de mesure, Fréquence d'échantillonnage, Unité).
- **SIMULATION :** Le logiciel doit permettre de définir la durée totale de la simulation et le pas de temps, et de générer les valeurs nominales à chaque itération.
- **ANOMALIE :** Le système doit permettre d'injecter des défauts maîtrisés sur les composants pour dégrader leur comportement de manière réaliste.
- **EXPORT :** Les données issues de la simulation doivent être exportables au format CSV pour permettre l'entraînement de modèles d'IA.

3.2 Exigences d'Interface et Non-Fonctionnelles

- **UI :** L'interface doit être ergonomique, utilisant des panneaux ancrables (docks) pour séparer l'explorateur de système, les propriétés et la visualisation des résultats.
- **TECH :** Le développement doit être réalisé en Python Orienté Objet (POO) en respectant l'architecture MVC (Modèle-Vue-Contrôleur). Le projet doit être versionné sous Git.

4. Conception et Maquettage

4.1 Maquettage de l'IHM

L'interface graphique a été conçue pour s'apparenter à un environnement de développement ou de simulation industriel. Elle est divisée en trois zones principales :

1. **Un explorateur de système (à gauche) :** Affiche l'arborescence des composants et de leurs capteurs respectifs suite à l'importation d'un modèle.
2. **Un panneau de propriétés (à droite) :** Permet à l'utilisateur de configurer dynamiquement le composant ou le capteur sélectionné (modification de la fréquence, injection d'anomalie).
3. **Une zone centrale :** Destinée à afficher le journal d'exécution (logs) et les résultats de la simulation en temps réel.

4.2 Modélisation UML

4.2.1 Axe Fonctionnel (Cas d'utilisation)

Analyse du sujet

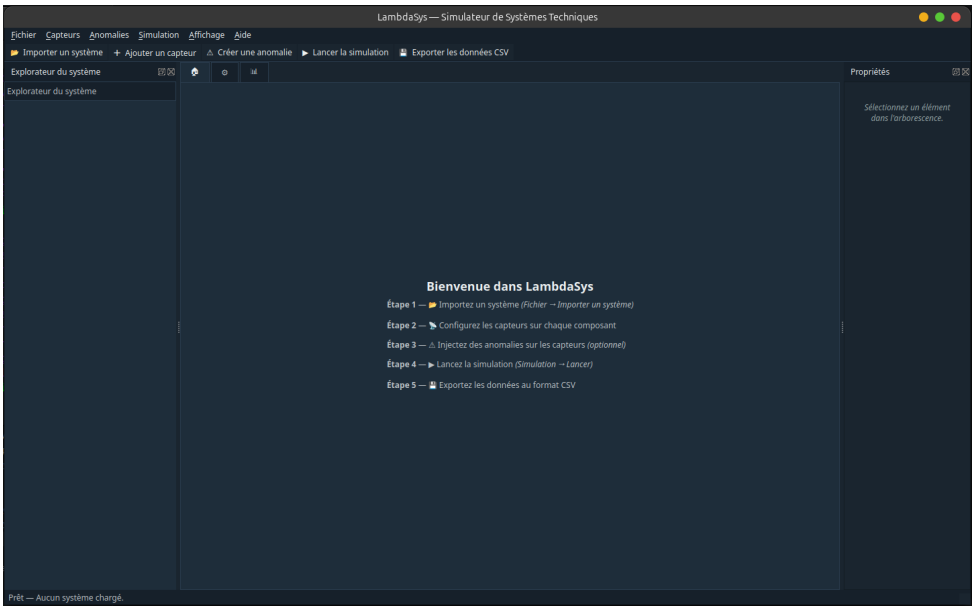

FIGURE 1 – Maquette de l'interface principale de LambdaSys

- **Acteur :**
  - **Utilisateur :** utilise le logiciel pour créer et gérer les simulations et les anomalies.
- **Système :**
  - **LambdaSys :** le système avec lequel interagit l'utilisateur, générant les données simulées et les anomalies.
- **Cas d'usage principaux :**
  - **Importer un système :** Permet de modéliser un système technique à partir d'un fichier ou d'une configuration existante.
  - **Gérer les capteurs :** Ajouter, modifier, déplacer ou supprimer un capteur ; Importer / enregistrer la configuration d'un capteur.
  - **Gérer les anomalies :**
    - Créer une anomalie (Paramétrer ou création aléatoire).
    - Supprimer une anomalie.
  - **Gérer la simulation :** Lancer, mettre en pause ou arrêter une simulation ; Enregistrer les données produites.

Diagramme de cas d'usage

Le diagramme des cas d'utilisation met en évidence les interactions de l'utilisateur (Ingénieur) avec le système. Les cas d'utilisation principaux sont : "Importer un système", "Configurer les capteurs", "Lancer la simulation" et "Exporter les données (CSV)".

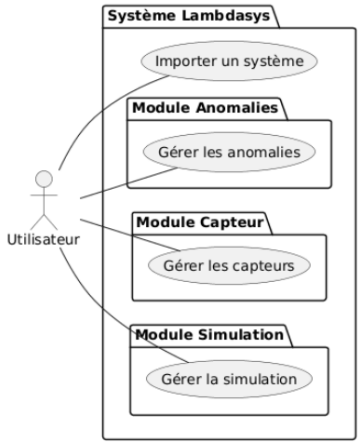

FIGURE 2 – Diagramme des cas d'usages simple

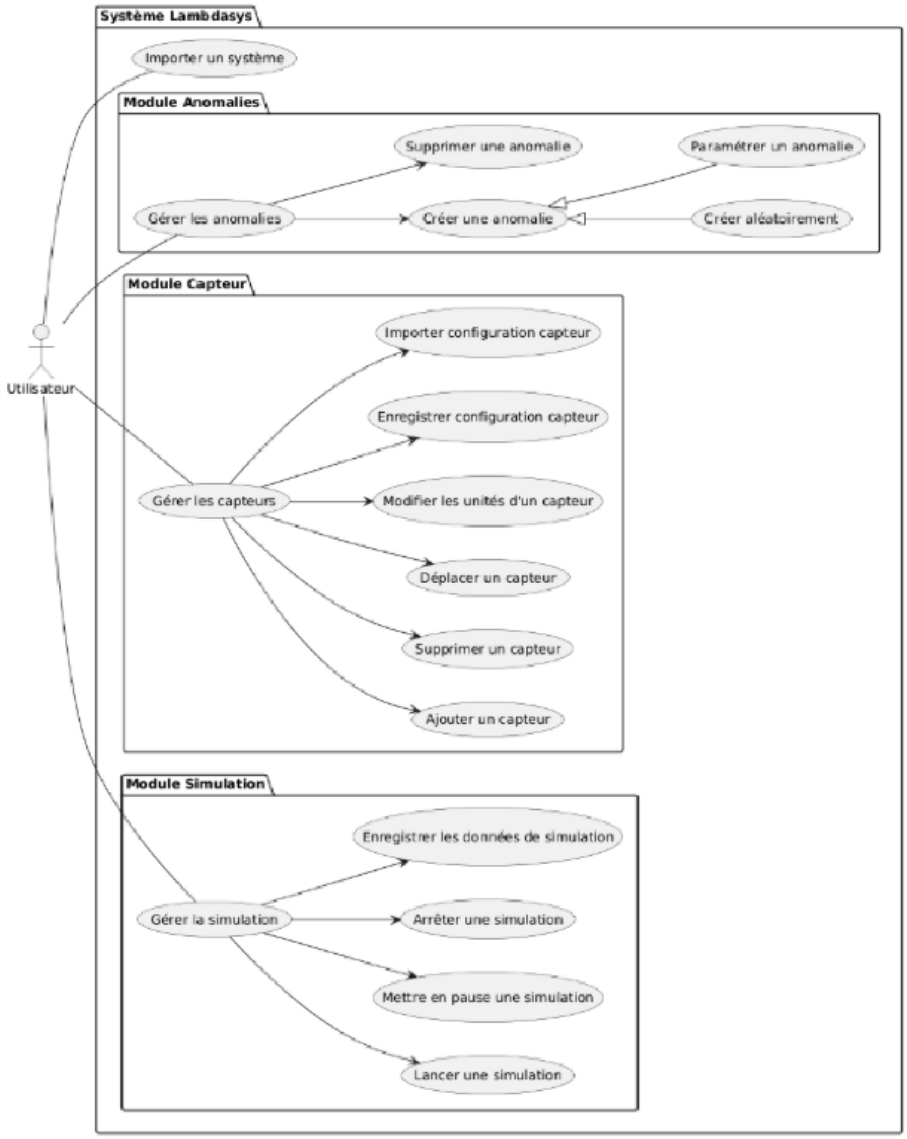

FIGURE 3 – Diagramme des cas d'usages developpé

Sénario nominal : Gérer une simulation

Le scénario suivant détaille le déroulement nominal d'une simulation au sein du logiciel :

1. L'utilisateur accède à la fonctionnalité gérer une simulation.
2. Le système affiche les options disponibles (lancer une simulation, mettre en pause, arrêter, enregistrer les données).
3. L'utilisateur choisit de lancer une simulation.
4. Le système initialise les paramètres de simulation (temps, capteurs, conditions).
5. Le système démarre la simulation.
6. Le système affiche en temps réel les résultats (données des capteurs, états du système).
7. La simulation s'exécute jusqu'à la fin du temps de simulation.
8. Le système met fin à la simulation.
9. Le système propose à l'utilisateur d'enregistrer les résultats.
10. L'utilisateur choisit d'enregistrer les résultats de la simulation.
11. Le système sauvegarde les résultats de la simulation.

Scénarios alternatifs (A)

- **A1 – Mettre en pause la simulation** (commence à l'étape 6) :
  1. L'utilisateur met la simulation en pause.
  2. Le système suspend la simulation.
  3. L'utilisateur peut : reprendre la simulation (retour étape 6), enregistrer les données (étape 11) ou arrêter la simulation (étape 9).
- **A2 – Arrêter la simulation** (commence à l'étape 6) :
  1. L'utilisateur arrête la simulation.
  2. Le système arrête la simulation.
  3. Le système propose d'enregistrer les résultats (reprendre à l'étape 9).
- **A3 – Ne pas enregistrer les résultats** (commence à l'étape 9) :
  1. L'utilisateur choisit de ne pas enregistrer les résultats.
  2. Le système termine le processus.

Exceptions (E)

- **E1 – Erreur lors du lancement de la simulation** (commence à l'étape 3 ou 4) :
  1. Le système détecte une erreur (paramètres invalides ou capteurs non configurés).
  2. Le système affiche un message d'erreur.

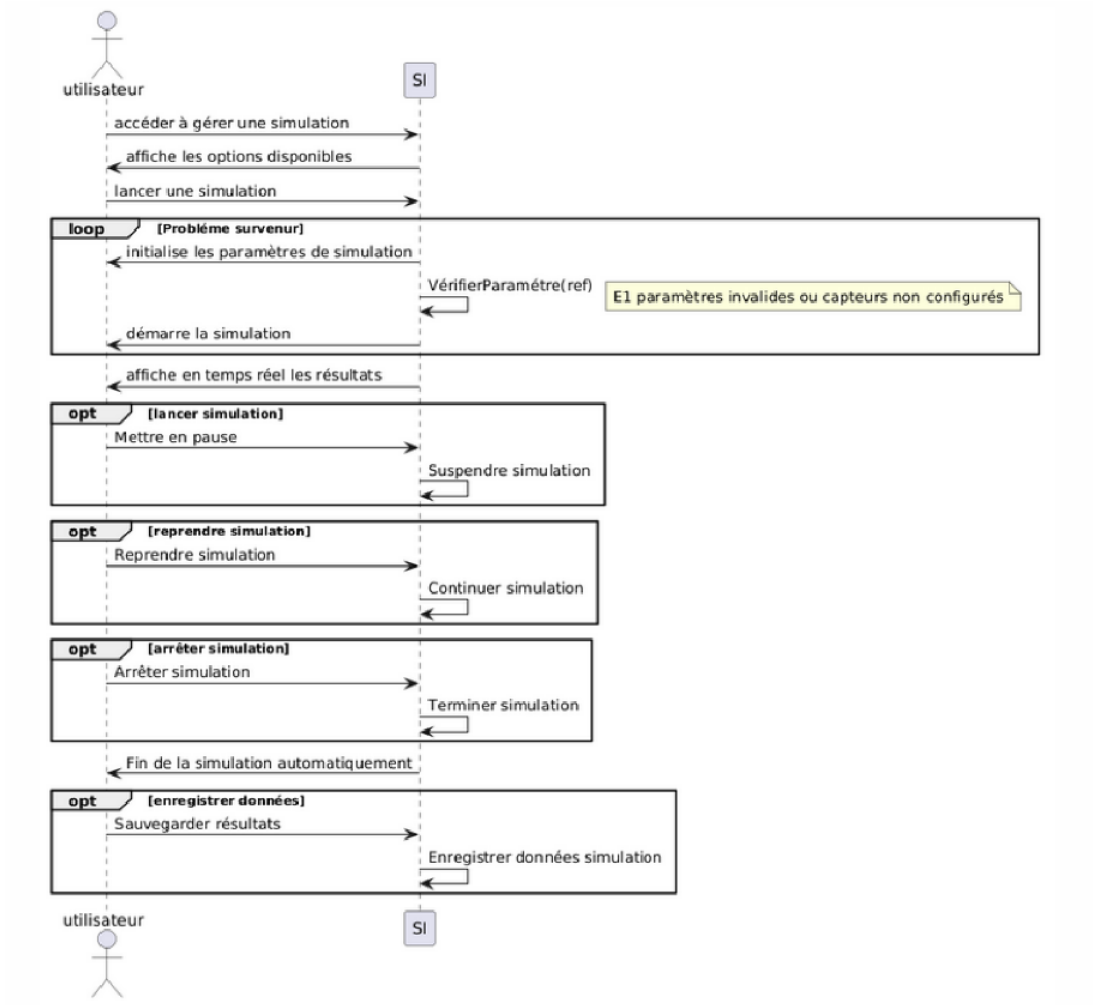

FIGURE 4 – Diagramme de séquence : Exécution d'une simulation

Note : La même démarche d'analyse a été réalisé pour le cas d'usage « Gérer une anomalie ». Pour ne pas alourdir le rapport, les détails et le diagramme de séquence correspondant sont disponibles en Figure 7 en Annexe.

4.2.2 Axe Statique (Diagramme de Classes)

L'axe statique repose sur un motif d'architecture strict : le **MVC (Modèle-Vue-Contrôleur)**.

- **Le Modèle** est composé d'une classe abstraite mère générique (`Component`) dont héritent toutes les pièces du système technique importé.
- **La Vue** est gérée via la bibliothèque PyQt5 (`MainWindowGUI`), s'occupant exclusivement de l'affichage.
- **Le Contrôleur** (`MainWindowController`) orchestre les actions de l'utilisateur et interagit avec le moteur de simulation (`SimulationEngine`).

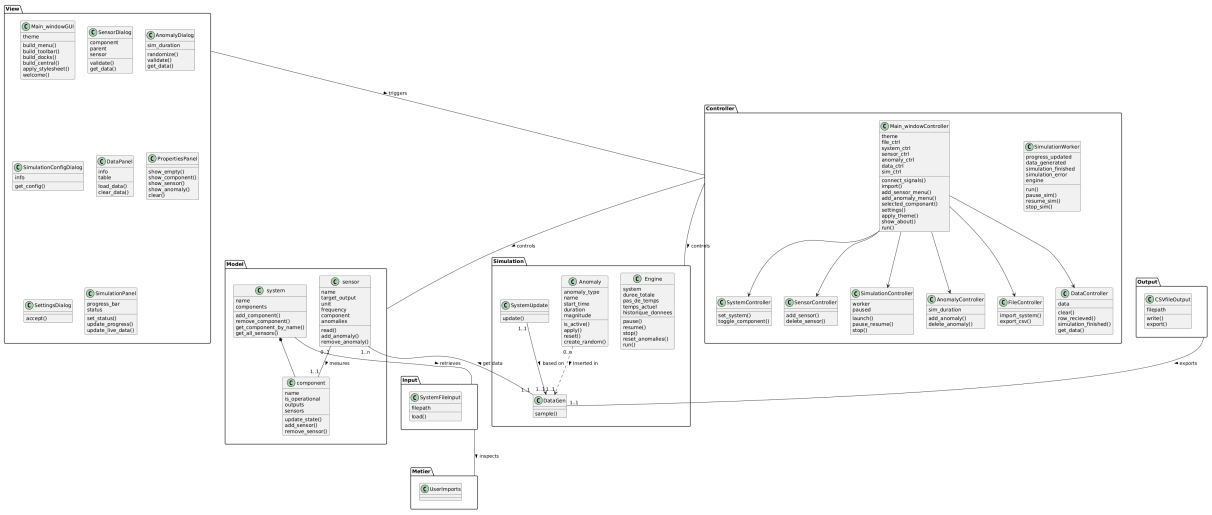

FIGURE 5 – Diagramme de classes détaillant l'architecture MVC

4.2.3 Axe Dynamique (Diagramme de Séquence)

Le diagramme de séquence illustre le scénario nominal de la gestion d'une simulation. Il détaille la boucle temporelle : pour chaque pas de temps, le contrôleur demande au modèle de calculer son nouvel état physique, puis récupère les valeurs des capteurs pour les stocker en mémoire avant de mettre à jour l'interface.

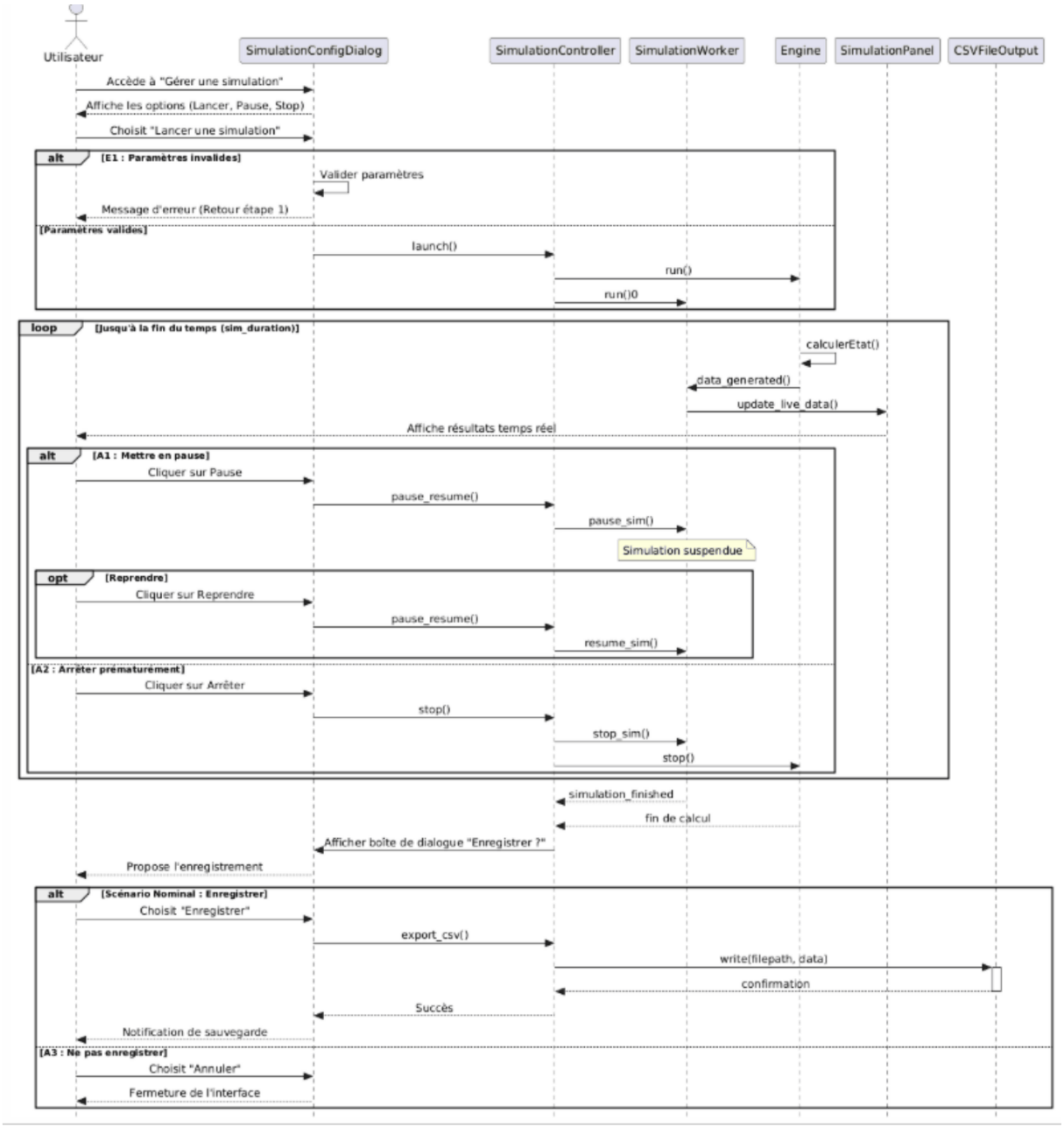

FIGURE 6 – Diagramme de séquence dynamique "Gérer une simulation"

Note : Un Diagramme de séquence dynamique a été réalisée pour le cas d'usage « Gérer une anomalie ». Pour ne pas alourdir le rapport, les détails et le diagramme de séquence correspondant sont disponibles en Figure 8 en Annexe.

5. Implémentation et Bilan Technique

5.1 Choix techniques : Introspection et Généricité

Pour répondre à la contrainte stricte de concevoir un logiciel "générique", nous avons fait le choix d'utiliser l'introspection en Python (via les bibliothèques natives `importlib` et

`inspect`).

Plutôt que de coder les composants techniques (pompe, valve, moteur) en dur dans le logiciel, LambdaSys charge dynamiquement des scripts `.py` externes fournis par l'utilisateur. Le logiciel inspecte le contenu du fichier à la volée, identifie les classes héritant de notre classe de base, et construit l'arborescence de l'interface graphique automatiquement. Cela rend l'outil adaptable à n'importe quel domaine d'ingénierie.

5.2 Problèmes rencontrés et solutions

Durant la phase de développement, plusieurs défis techniques se sont présentés :

- **Débordement de l'interface graphique :** Lors des premiers tests de simulation, l'affichage du dictionnaire de données dans un simple `QLabel` provoquait un étirement non désiré de la fenêtre. Ce problème a été résolu en utilisant un `QTextEdit` qui gère nativement les barres de défilement.
- **Environnement de développement Linux :** Des conflits de paquets (PEP 668) lors de l'installation de PyQt5 ont été contournés par la mise en place rigoureuse d'environnements virtuels (`venv`), garantissant ainsi la portabilité de notre code sur différentes machines.

6. Difficultées rencontrées

Au cours du projet, nous avons du faire face à différents problèmes. Ces problèmes allant du plus basique aux plus complexes nous ont permis d'apprendre à mieux structurer le projet et à éviter des incohérences qui auraient pu nous bloquer durant la conception du projet. Cela commence dès le depart lorsqu'il fallait réflechir à la façon dont on allait aborder le problème.

Une fois avoir bien identifié les differentes tâches à faire vient la partie UML qui n'a pas été simple mais qui ne nous a pas posé de grandes difficultés pour autant. L'axe statique a été le plus contraignant, notamment la partie sur le diagramme de classe qui s'est avèrée particulièrement difficile car il fallait très souvent revenir dessus lorsque l'implémentation en Python devenait fastidieuse.

En ce qui concerne la partie Implementation, il s'agit de la partie la plus complexe pour plusieurs raisons. La première étant la structure du code car le fait d'avoir un fichier par classe rend certe le code très polyvalent mais devient très difficile à manipuler surtout lorsque l'on souhaite faire des modifications. Il était donc important de bien séparer les différents fichier et surtout de bien les structurer.

Nous avons également rencontré des difficultés sur les anomalies et la simulation. Les anomalies étaient mal intégrées au projet et pouvaient parfois faire planter la simulation, un problème qui a été corrigé rapidement. La simulation a reçu énormement de modifications, ce qui a nécéssité plusieurs de jour de travail pour avoir un résultat idéal.

Pour le reste du projet nous n'avons rencontré aucune difficulté significative.

7. Conclusion

Le projet LambdaSys a permis de concevoir avec succès un logiciel de simulation modulaire et générique. En respectant une méthodologie Agile rigoureuse et en s'appuyant sur une conception UML solide, nous avons pu implémenter une architecture MVC robuste en Python.

La première phase du cahier des charges est pleinement validée : l'utilisateur peut importer un modèle externe, configurer des capteurs, lancer une simulation temporelle et exporter les données nominales générées au format CSV. Ces jeux de données, garantis sans biais non maîtrisés, constituent désormais une base solide et exploitable pour la future étape du projet : l'entraînement et la validation de modèles d'Intelligence Artificielle pour la maintenance prédictive.

8. Annexe

Scénario nominale : Gérer les anomalies
1- L'utilisateur sélectionne l'option "Gérer les anomalies".
2- Le système présente l'état actuel des anomalies.
3- L'utilisateur choisit "Créer une anomalie".
3- L'utilisateur choisit "Paramétrer une anomalie".
4- L'utilisateur saisit les paramètres requis (type, intensité, durée).
5- Le système valide les paramètres et enregistre l'anomalie.
6- L'anomalie apparaît dans le tableau de bord de simulation.

Gérer les anomalies - scénarios alternatifs :
A1 – L'utilisateur choisit "Créer aléatoirement"
(commence à l'étape 3)
1- L'utilisateur choisit "Créer aléatoirement".
2- Le systèmegénére une anomalie aléatoirement.
(retour à l'étape 6)

E1 – Conflit de paramètres : Les valeurs saisies par l'utilisateur sont physiquement impossibles
(commence à l'étape 3 ou 4)
1- Le système détecte une incohérence des valeurs.
2- Le système affiche un message d'erreur.
3- L'utilisateur corrige les paramètres.
Le système retourne à l'étape 1.

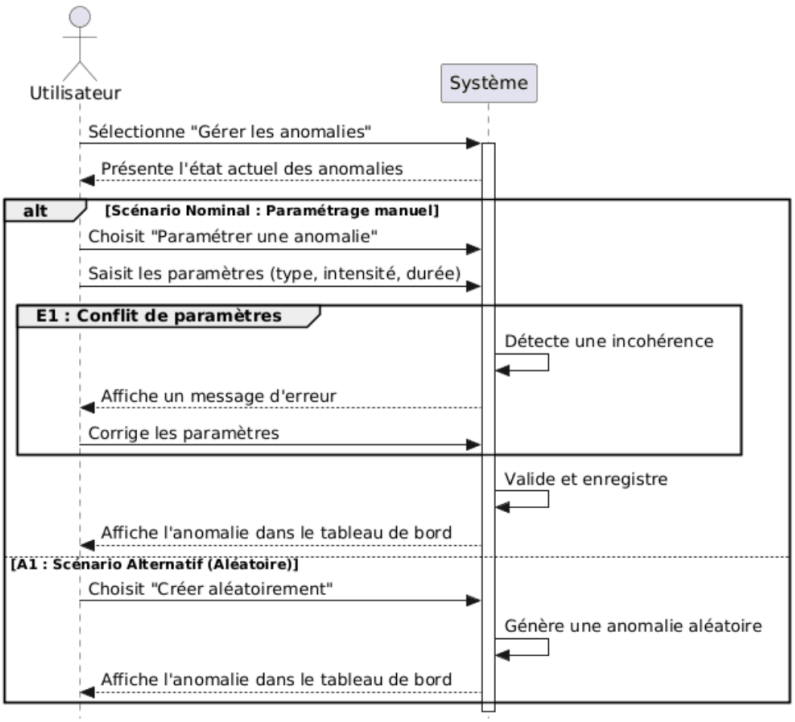

FIGURE 7 – Sénario nominame et Diagramme de séquence de "Gérer les anomalies"

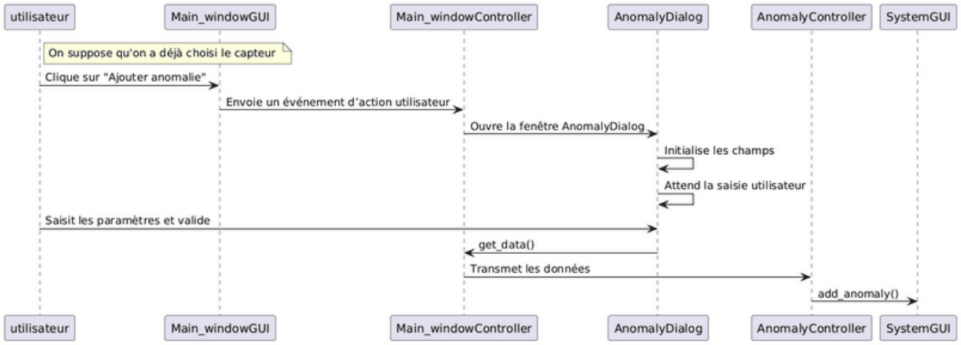

FIGURE 8 – Diagramme de Séquence Dinamique de "Gérer les anomalies